

Ejercicios evaluables UA6

Esta actividad aborda los criterios de evaluación del RA.

Instrucciones

- Todas las actividades que se realicen deben entregarse mediante un enlace al repositorio de Github y un documento (si se pide alguna evidencia o se plantean preguntas): DAW_DWES-UA6-ApellidosNombre.pdf
- Las entregas deberán realizarse en la tarea correspondiente en el formato especificado antes de la hora de finalización. Cualquier tarea que no cumpla estos requisitos tendrá una calificación de 0.
- Se pedirá que defiendas tu práctica. Si no puedes justificar tu trabajo o responder las preguntas que se te plantean, la calificación de la tarea será de 0.

Sobre el uso de IA y fuentes externas

Puedes utilizar herramientas como buscadores o inteligencia artificial como apoyo, pero no como sustituto de tu trabajo. Las respuestas deben estar redactadas por ti, con tus propias ideas y razonamientos.

Se valorará especialmente el análisis personal y la justificación razonada.

Ejercicio 1

Hasta ahora, la biblioteca era común y no pertenecía a nadie en concreto. En este ejercicio, la idea es dar el primer paso hacia una plataforma real, donde cada juego tenga dueño.

Aprenderás a adaptar el sistema para que las entradas existentes sigan funcionando y, al mismo tiempo, pasen a estar asociadas a un usuario.

Dónde se hace

- ☐ Modificación de modelo + migración en base de datos

Qué cambios se deben hacer

- ☐ En primer lugar debes modificar el modelo `LibraryEntry` y añadir un nuevo campo `user`, que será una clave foránea al modelo `User`. ¿Qué relación (1:1, 1:N, N:M) hay entre `LibraryEntry` y `User`?
 - Una vez que modifiques el modelo, ¿qué pasa con los datos existentes? Debes considerar si añades `null=True`, o creas un usuario *demo* y asocias esas entradas al usuario.
 - No te olvides de crear y aplicar la migración.

Comprobaciones que debes cumplir

- ☐ No se pueden borrar los datos existentes.

Qué debes probar

- ☐ El proyecto arranca sin errores con la nueva migración aplicada.
- ☐ Debes realizar el resto de ejercicios para poder crear nuevas entradas y asociarlas a los usuarios

Ejercicio 2

Para que cada persona tenga su propia biblioteca, primero necesita una cuenta. En este ejercicio se crea el punto de entrada para que los usuarios puedan registrarse en la plataforma.

El backend debe comprobar que los datos tienen sentido y explicar claramente qué ocurre cuando algo falla.

Dónde se aplica

- ☐ Ruta: `POST /api/auth/register/`

Qué datos envía el frontend

El frontend debe mandar JSON con estos campos:

- ☐ `username (string)`
- ☐ `password (string)`

Validaciones que debes hacer antes de aceptar

Tu backend debe revisar los datos que recibe:

- ☐ Están los campos obligatorios y con tipos correctos
- ☐ username no está ya en uso
- ☐ password tiene mínimo 8 caracteres

Respuesta si todo va bien

Si la petición es correcta, se crea el usuario y se devuelve:

- ☐ Código: 201
- ☐ Body: { "id": 1, "username": "ana" }

Respuesta si algo falla

Si hay cualquier problema de validación :

- ☐ Código: 400
- ☐ Body: JSON con error: validation_error, con el mismo formato que establecimos en la semana 1

¡Importante!

- ☐ No debes devolver la contraseña nunca

Qué debes probar

- ☐ 1 registro correcto (201)
- ☐ Al menos 4 registros inválidos (400) cubriendo errores distintos: username repetido, password corta, tipos incorrectos, campos ausentes, JSON vacío...

Ejercicio 3

Una vez registrado, el usuario necesita identificarse para que el sistema sepa quién es en cada momento. En este ejercicio se trabaja cómo iniciar sesión y cómo el backend recuerda al usuario entre peticiones.

Gracias a esto, el sistema puede responder de forma distinta según quién esté usando la plataforma. Es el paso clave para dejar de tratar a todos los usuarios como si fueran el mismo.

Dónde se hace

- ☐ Ruta login: POST /api/auth/login/
- ☐ Ruta comprobación: GET /api/users/me/

Cómo deben funcionar las rutas

- En el login:
 - Se debe enviar JSON con un username y un password, ambos como string.
 - Si las credenciales son correctas, se debe devolver una respuesta con código 200 y contenido igual al que establecimos la semana 1.
 - El sistema queda en estado autenticado.
 - Si las credenciales son incorrectas, se debe devolver una respuesta con código 401 y un error: unauthorized, con message: Credenciales incorrectas. Si por el contrario faltaran campos o los tipos son incorrectos, código 400 y error: validation_error.
- En el de comprobación:
 - Se debe comprobar que el usuario está autenticado.
 - Si lo está, devolver un código 200 y la información (id y username) en JSON.
 - Si no lo está, devolver un código 401 y error: unauthorized con message: No autenticado.

Qué debes probar

- Login correcto (200)
- Login incorrecto (401)
- GET /api/users/me/ sin autenticar (401)
- GET /api/users/me/ después de login correcto (200)

Ejercicio 4

Ahora que el sistema sabe quién es cada usuario, toca proteger la información. En este ejercicio se consigue que cada persona vea únicamente su propia biblioteca. El backend debe comportarse como una plataforma real: sin mostrar datos ajenos ni dar pistas sobre ellos.

Dónde se hace

- Ruta listado: GET /api/library/entries/
- Ruta detalle: GET /api/library/entries/{id}/

Comportamiento esperado en cualquiera de las dos rutas

- Si no está autenticado: se debe devolver una respuesta con código 401 y un error: unauthorized, con message: No autenticado.
- Si está autenticado:
 - El listado devuelve solo las entradas del usuario autenticado.
 - El detalle devuelve la entrada solo si pertenece al usuario autenticado.

Si un usuario intenta acceder a una entrada que no es suya

Para no revelar existencia de recursos ajenos, se devuelve una respuesta con código 404 y body:

```
{
  "error": "not_found",
  "message": "La entrada solicitada no existe"
}
```

Restricción importante

- ☐ No exponer datos de otros usuarios

Qué debes probar

- ☐ Crear 2 usuarios con entradas distintas
- ☐ Probar que cada usuario ve solo su biblioteca
- ☐ Probar que acceder al id de otra persona devuelve 404 con el JSON especificado.

Ejercicio 5

El usuario añade un juego a su biblioteca y, con el tiempo, va cambiando su estado y las horas jugadas. En este ejercicio todo encaja: el sistema sabe quién es el usuario, valida los datos y comprueba los permisos.

Cada acción queda asociada automáticamente a la persona correcta, sin que el cliente tenga que indicarlo.

Dónde se hace

- ☐ Crear: POST /api/library/entries/
- ☐ Actualizar: PATCH /api/library/entries/{id}/

Nuevas reglas para cada ruta

- ☐ En crear:
 - Cada entrada debe asociarse con el usuario que hace la petición, no con el usuario que se envía desde los datos.
 - Si no está autenticado, se devuelve un código 401, con error: unauthorized y mismo formato que en el ejercicio 3.
- ☐ En actualizar:
 - Se debe comprobar que el usuario que envía la petición es el dueño de la entrada.
 - Si no lo es, se devuelve un código 404 con error: not_found, con el mismo formato que el ejercicio 4.
 - Si no está autenticado, se devuelve un código 401, con error: unauthorized y mismo formato que en el ejercicio 3.

Qué debes probar

- ☐ Como usuario A: crear entrada y listar
- ☐ Como usuario B: intentar ver/modificar la entrada de A y comprobar el 404
- ☐ Actualización válida (200) y varias inválidas (400)
- ☐ Operaciones sin autenticar (401)

Rúbrica de evaluación de la tarea

Criterio de evaluación	Nivel alto (9-10)	Nivel medio (5-9)	Nivel Bajo (1-5)	No logrado (0)